LLM-Driven SystemVerilog Assertion Generation for Formal Verification

CS525 – Formal Methods for System Verification

Group 2
Pranjal 230101077
Dhruvkumar R Pansuriya 230101071
Shrish Uttarwar 230101108

IIT Guwahati · Dept. of Computer Science & Engineering · April 2026

Problem Statement

Primary Research Question:

How can an automated system synthesise information from ambiguous, multi-page natural-language specifications and complex RTL code to generate SVAs that are both syntactically correct and functionally meaningful — within the token and cost constraints of modern LLMs?

Problem Statements & Task Objectives

P1. Long-Context Limits — selecting a minimal, sufficient context subset.

P2. Agent Memory — accumulating and reusing verified knowledge across steps.

P3. Cloud Communication Efficiency — reducing API token and latency overhead.

P4. Redundancy in Multi-Agent Systems — minimizing duplicated computation and assertions.

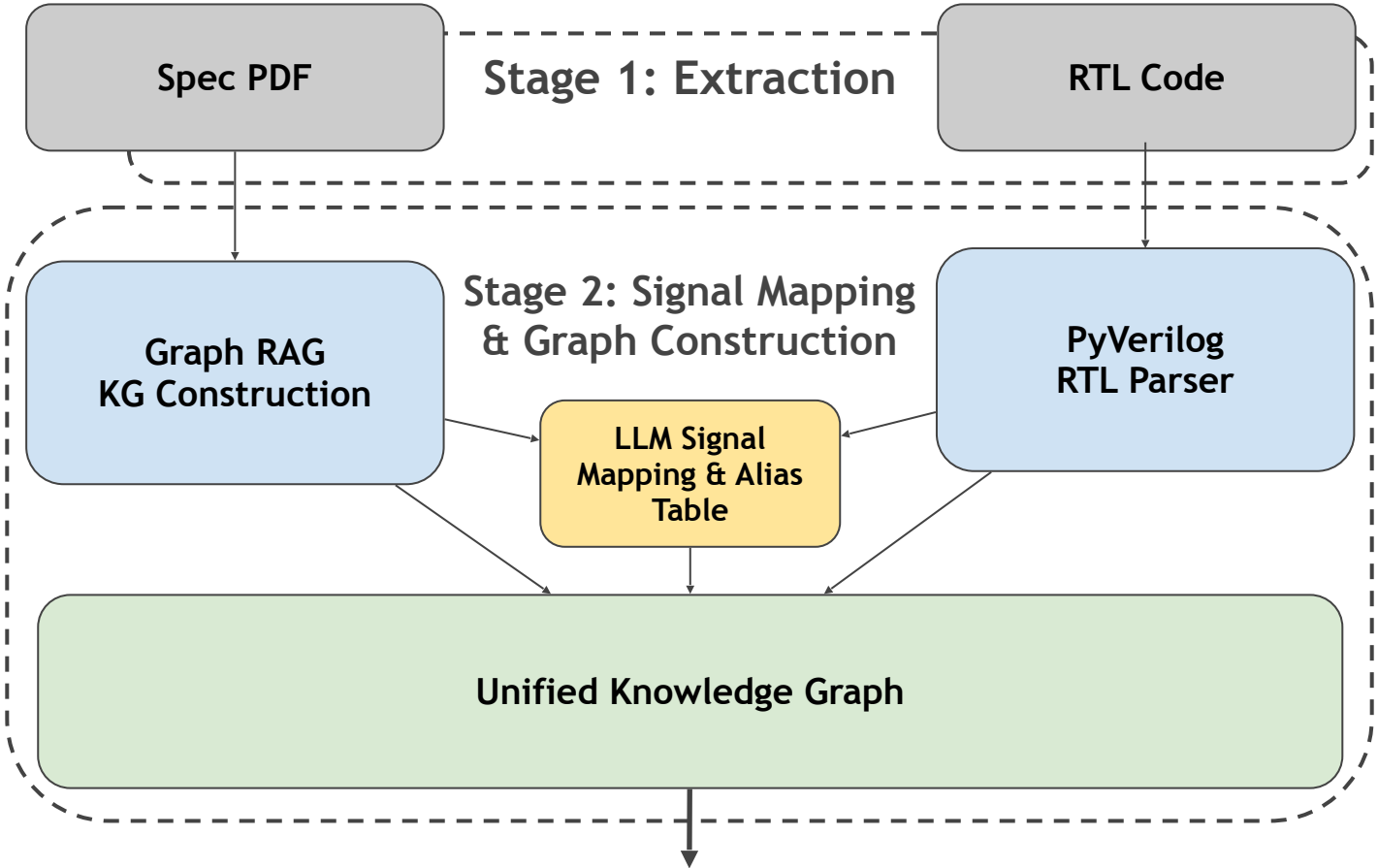
P5. Feedback-Driven Refinement — improving outputs using verifier or critic feedback.

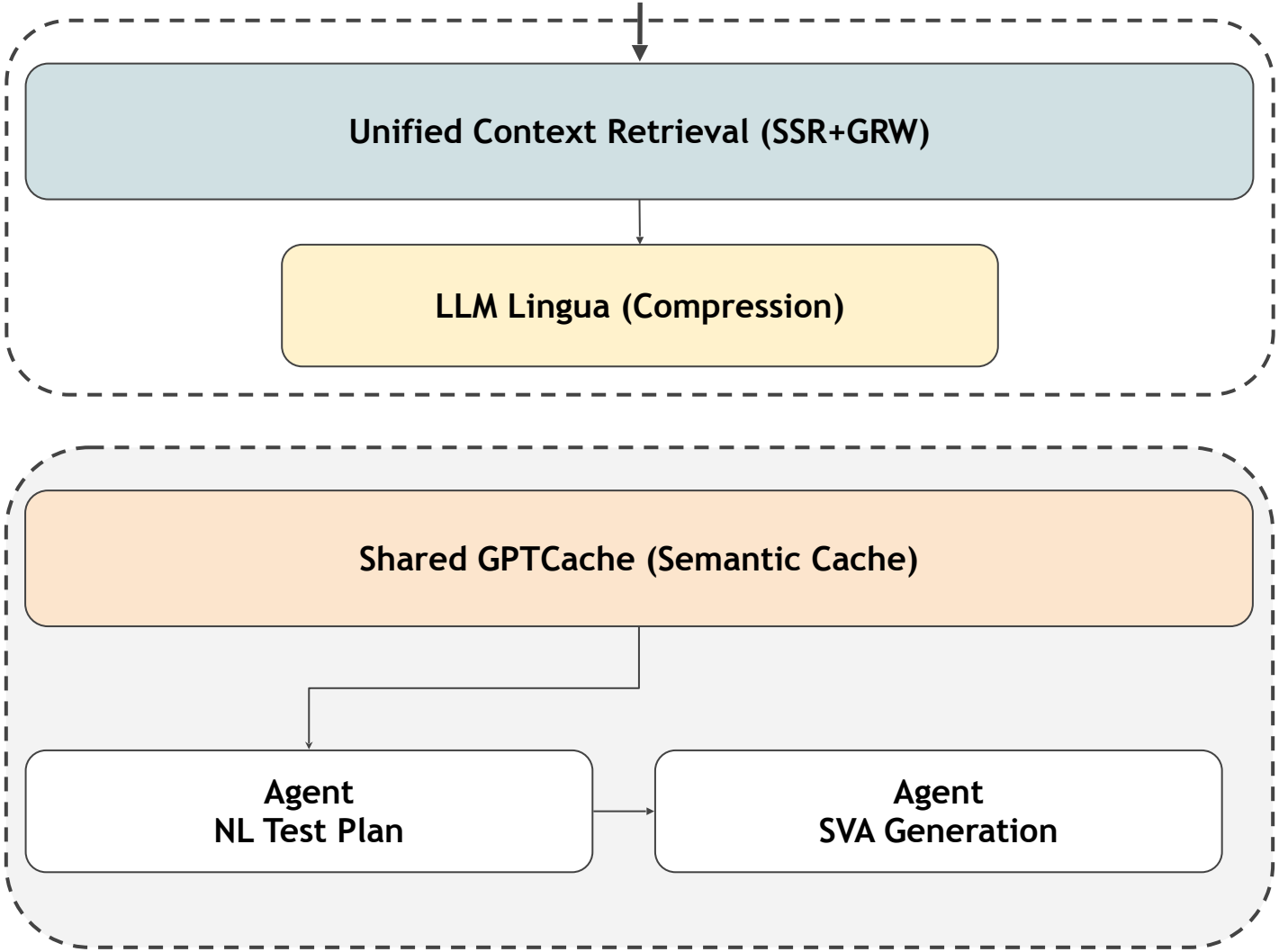
Our Solution

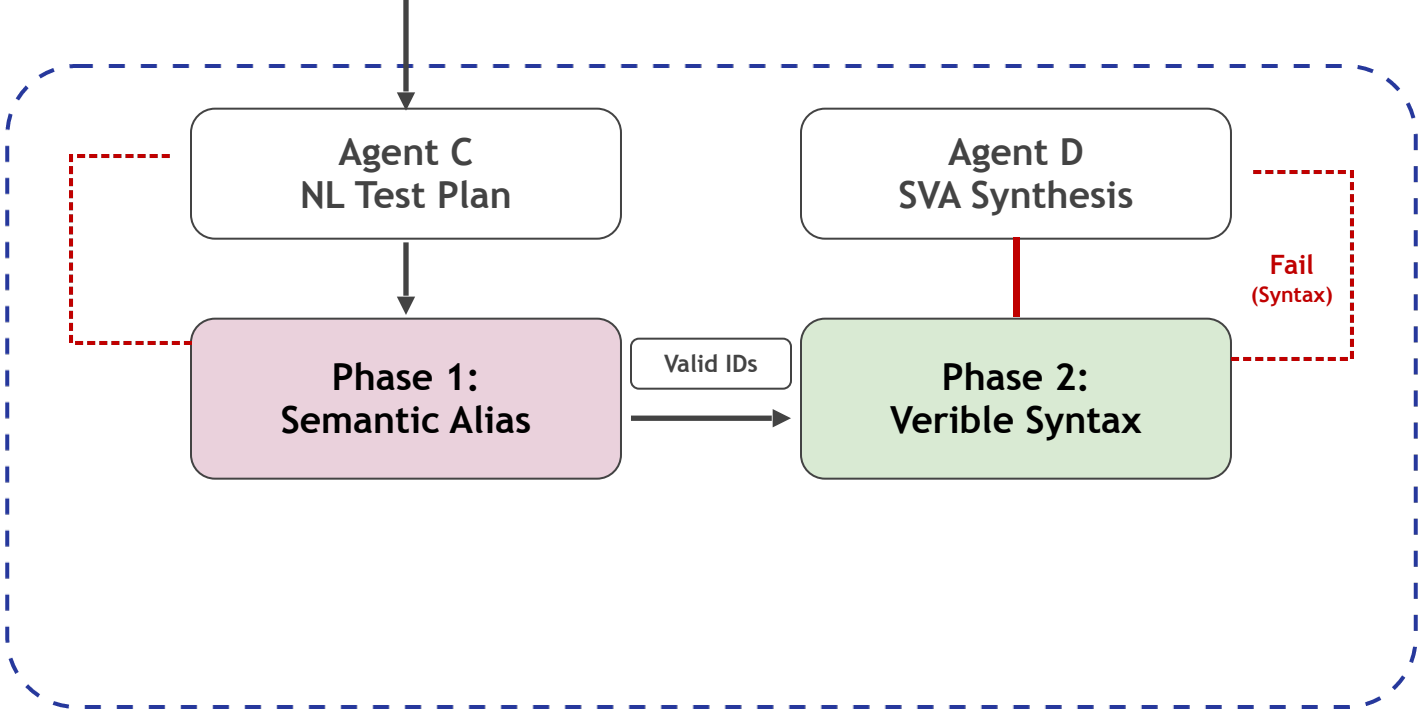
- *"Turning* ambiguous specs + complex RTL *into precise assertions using* LLMs + structured reasoning*."*

- "Efficient, scalable, and feedback-driven—*built to handle long context, redundancy, and real-world constraints."*

Knowledge Graph (KG)




Unified Context Retrieval (SSR+GRW)
LLM Lingua (Compression)
Shared GPTCache (Semantic Cache)
Agent
NL Test Plan
Agent
SVA Generation



Semantic Alias: Matches generated signal names with valid RTL identifiers to detect hallucinated or incorrect signals.

Verible Syntax: Checks whether the generated SVA follows correct SystemVerilog syntax and reports syntax errors.

System Architecture — Full Pipeline

Stage 1

Extraction & Alignment

Spec PDF + RTL Verilog

GraphRAG KG (Spec-side)

PyVerilog (RTL-side)

Signal Mapping & Alias Table

Unified KG G = (V, E)

Stage 2

Context Synthesis & Compression

Global Summarisation

Signal-Specific Retrieval (SSR)

LLMLingua Compression

GPTCache Routing

Stage 3

SVA Generation

Agent C NL Test Plan

Agent D SVA Synthesis

Groq API (Fast Format)

Cache Deduplication

Stage 4

Checking & Repair

Phase 1 Alias Validation

Phase 2 Verible Syntax

LLM Repair Loop

Accepted SVAs .sva files

Diagnostics JSON Reports

Stage 1 — Knowledge Graph Construction

Spec-Side: GraphRAG + Domain Schema

Entity Types (E_t):

Module · Signal · Port · Register · FIFO · Clock · Interrupt · FSM · Protocol · Constraint

Relation Types (R_t):

connectsTo · defines · triggers · dependsOn · HasSignal · UsesClock · DescribesOperation · transmitsData · corresponds_to

RTL-Side: PyVerilog Parser

- Module structure — AST traversal; ports (direction, width); module instantiations
- KG refinement — RTL info merged into G_0 to enrich spec-side graph

Spec KG + RTL KG → Unified Knowledge Graph G = (V, E)

Signal Mapping — Bridging Spec & RTL

"system reset" in spec ↔ nRST / rst_n in RTL — without alignment, generated SVAs reference non-existent signals

1. **Input Extraction**
   Spec entities from KG node types (SIGNAL, PORT, REGISTER, CLOCK)
   RTL module ports & internal signals from PyVerilog

2. **LLM Semantic Alignment**
   LLM receives mentions + RTL signals + local spec context
   Outputs JSON: canonical_name · rtl_name · confidence · active_low · match_method

3. **Alias Table Construction**
   Fast lookup: RTL signal → canonical name
   Normalises naming variants; basis for fuzzy fallback matching

4. **Confidence Gating (≥ 0.7)**
   HIGH: inject corresponds_to edges into Unified KG
   LOW: string-based substring similarity → references edges

Key Artifacts

- signal_aliases.json
- alias_lookup_table.json
- signal_mapping_decisions.json
- alignment_full_prompt.txt
- signal_mapping_overview.json

Stage 3 — SVA Generation

Two-Step Generation via Groq

Step 1

Natural Language Test Plan

LLM generates human-readable verification intents from pruned context

e.g. "Verify that TX_EMPTY asserts when the FIFO count reaches zero"

→ nl_plans.txt

Step 2

SVA Synthesis

Each NL plan → formal SystemVerilog Assertion
Prompt includes correct SVA syntax examples as few-shot guidance

e.g. @(posedge PCLK) ((PADDR >= 24) && (PADDR <= 31)) && (PENABLE) |-> ((PADDR >= 0) ...

→ property_goldmine_N.sva

KG as Robust Agent Memory (P2)

Traditional RAG fails when causal hardware logic is fragmented across semantic chunks.

RQ1.4

Memory Representation & Fidelity

Structured entity-relation schemas (E_t, R_t) outperform flat embeddings.
KG enforces relational memory — updates are edge insertions/attribute changes, not vector dilution.
Causal links (clock-domain crossings, nested resets) preserved reliably.

RQ1.6

Task-Adaptive Memory Granularity

Broad Test Planning (Agent C): coarse-grained SSR at 800+ token chunks.
Precise SVA Synthesis (Agent D): GRW-AS extracts exact causal chains (TX_EMPTY → apb → PRESETn).
LLMLingua applies high compression to verbose spec memory, keeps RTL memory intact.

Memory structure: Spec Vectors (semantic) + RTL AST (structural) → fused inside Unified KG

LLMLingua — Prompt Compression (P1 + P3)

Coarse-to-fine prompt compression: small LM computes token perplexity → prunes low-information tokens → compressed prompt to GPT-4o / Groq.

Overall Statistics

Tokens w/o LLMLingua: 4616
Tokens w/ LLMLingua: 3876
Total Reduction: 16.03%

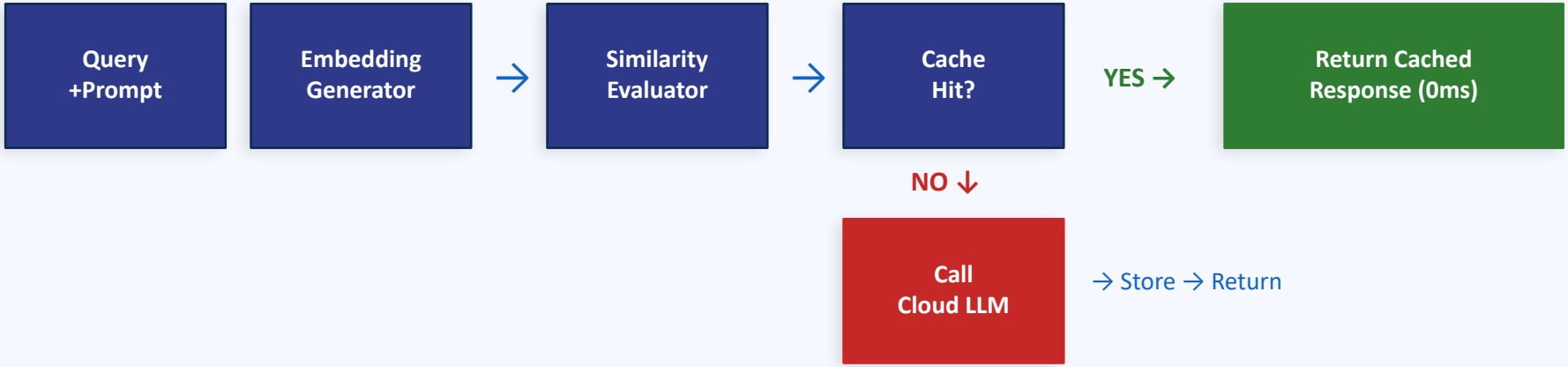
Detailed Breakdown

Dynamic NL Generation
Origin: 2163 | Compressed: 1807

Dynamic SVA Generation
Origin: 2453 | Compressed: 2069

Multiagent Orchestration with GPTCache (P3 + P4)

Eliminates redundant cloud API calls by storing and retrieving semantically similar prompt responses.



Problems Addressed:

- RQ1.7 — Minimizes information transfer; cached responses bypass full prompt re-transmission
- RQ2.2 — Early redundancy detection; intercepts repeated queries before reaching LLM
- RQ2.4 — Prompt mapping via embedding-based similarity; reuses results above threshold
- RQ2.5 — Flexible caching granularity: prompt-level or chunk-level
- Cache key = hash(agent_id + prompt_version + exact context) → prevents cross-agent contamination

Multiagent Orchestration with GPTCache (P3 + P4)

Task decomposition + specialisation to improve assertion quality while maintaining cloud efficiency.

Agent A	Agent B	Agent C	Agent D	Agent E
Spec Understanding	RTL Grounding	Test Planner	SVA Synthesizer	Referee / Checker
→Gemini	*→Gemini*	*→Groq*	*→Groq*	*→Gemini*

Two-Phase Assertion Quality Gate

Phase 1 (Loop A)

Semantic Alias Validation

[Phase 1]

Extract candidate identifiers from SVA
Compare with valid alias lookup table
One repair attempt if invalid signals detected
Why default-off: strict pre-filter over-rejects on noisy/generated corpora

Phase 2 (Loop B)

Verible Syntax Check

[Phase 2]

Wrap assertion in minimal valid SV module using RTL signals
Run verible-verilog-syntax binary
One repair attempt on failure

Outputs: accepted/ · discarded/ · reports/summary.json · reports/detailed_results.json · reports/logs/

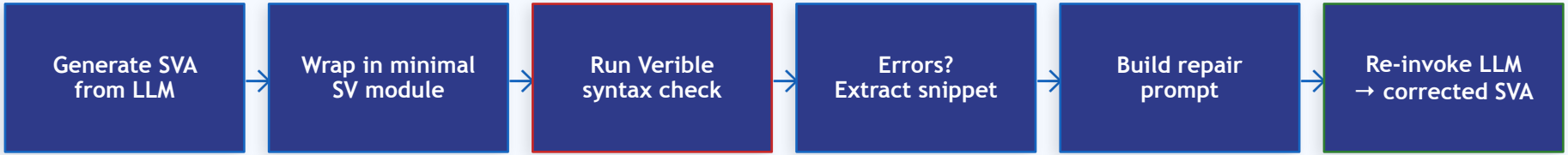
Verible Syntax Validation & Repair (P5)

Feedback-driven refinement loop using Verible — SystemVerilog-aware syntax checker.

Feedback Signals (RQ3.1):

- Binary: SVA passes / fails syntax check (#SynC)
- Structured: Verible stderr — precise diagnostics (e.g., missing clocking event, invalid operator)
- Localized: Line-level error → identifies exact failing assertion fragment

Repair Loop (RQ3.2):



Safeguards (RQ3.3):

Local correction per-instance · Bounded repair iterations · Context preserved across iterations · Syntax decoupled from semantic generation

Results

Generated Test Plans

nl_plans.txt

AssertionForge > logs > gen_plan_2026-03-21T04-20-04.109787_Mac_dhruvrpansuriya > nl_plans.txt

```
Signal PADDR:
Plan 1: PADDR: that when PADDR is within the range of 0 to 31, in the next cycle PADDR will remain within the valid address range.
Plan 2: PADDR: that the address signal PADDR has a value between 0 and 15, inclusive, 2 clock cycles after the reset signal PRESETn becomes deasserted, as indicated by t
Plan 3: PADDR: that if the address signal PADDR is within the range of 16 to 23 inclusive, then in the subsequent cycles, PADDR must continue to be within the range of 0
Plan 4: PADDR: that the address signal PADDR is within the range 0 to 7 inclusive, starting from three clock cycles after the reset signal PRESETn becomes deasserted, as
Plan 5: PADDR: that when PADDR is within the range of 24 to 31, in the next cycle PADDR will remain within the valid address range, as controlled by the signal PENABLE.
Plan 6: PADDR: that the address signal PADDR has a value between 8 and 15, inclusive, 4 clock cycles after the reset signal PRESETn becomes deasserted, as indicated by t
Plan 7: PADDR: that if the address signal PADDR is within the range of 0 to 7 inclusive, then in the subsequent cycles, PADDR must continue to be within the range of 0 t
Plan 8: PADDR: that the address signal PADDR is within the range 16 to 23 inclusive, starting from five clock cycles after the reset signal PRESETn becomes deasserted, a
Plan 9: PADDR: that when PADDR is within the range of 8 to 15, in the next cycle PADDR will remain within the valid address range, as controlled by the signal PENABLE an
Plan 10: PADDR: that the address signal PADDR has a value between 24 and 31, inclusive, 6 clock cycles after the reset signal PRESETn becomes deasserted, as indicated by

Signal PCLK:
Plan 11: PCLK: that when PCLK rises, if PENABLE is high and PSEL is high, then in the next clock cycle, PREADY will be high if the transfer is completed.
Plan 12: PCLK: that if PRESETn is low, then PCLK will not affect the state of PSLVERR until PRESETn becomes high.
Plan 13: PCLK: that when PCLK falls, if PWRITE is high, then in the next clock cycle, PRDATA will be invalid until PREADY becomes high.
Plan 14: PCLK: that after reset (PRESETn becomes high), PCLK will start generating clock edges, and PADDR will be stable after 2 clock cycles.
Plan 15: PCLK: that if PCLK is stable and PENABLE is high, then PSLVERR will be low if the transfer is successful.
Plan 16: PCLK: that when PCLK rises, if PSEL is low, then PENABLE will be ignored in the current clock cycle.
Plan 17: PCLK: that if PRESETn is high and PCLK is stable, then PADDR will be valid after 1 clock cycle.
Plan 18: PCLK: that when PCLK falls, if PENABLE is low, then PREADY will be low in the next clock cycle.
Plan 19: PCLK: that after 5 clock cycles of PCLK, if PWRITE is low, then PRDATA will be valid if PREADY is high.
Plan 20: PCLK: that if PCLK is not oscillating and PRESETn is high, then PSLVERR will be low after 3 clock cycles.

Signal PENABLE:
Plan 21: PENABLE: that when PENABLE is asserted and PSEL is high, PREADY will be set to 1 in the next clock cycle, as indicated by the clock signal PCLK.
Plan 22: PENABLE: that if PENABLE is low and PSEL is asserted, the state machine will move to the ACCESS phase in the next clock cycle, as determined by the clock signal
Plan 23: PENABLE: that when PENABLE is asserted and PADDR is within the valid range, PSLVERR will be set to 0, indicating no error, as checked by the clock signal PCLK.
Plan 24: PENABLE: that if PENABLE is asserted and PWRITE is high, the write transfer will complete at the end of the ACCESS cycle, as indicated by the clock signal PCLK
Plan 25: PENABLE: that when PENABLE is asserted and PSEL is high, PRDATA will be valid in the next clock cycle, as determined by the clock signal PCLK and the PREADY sig
Plan 26: PENABLE: that if PENABLE is low and PRESETn is asserted, the state machine will remain in the IDLE state, as checked by the clock signal PCLK.
Plan 27: PENABLE: that when PENABLE is asserted and PADDR is out of range, PSLVERR will be set to 1, indicating an error, as checked by the clock signal PCLK.
Plan 28: PENABLE: that if PENABLE is asserted and PSEL is low, the state machine will not move to the ACCESS phase, as determined by the clock signal PCLK.
Plan 29: PENABLE: that when PENABLE is asserted and PWRITE is low, the read transfer will complete at the end of the ACCESS cycle, as indicated by the clock signal PCLK
Plan 30: PENABLE: that if PENABLE is asserted and PSEL is high, PWDATA will be valid in the next clock cycle, as determined by the clock signal PCLK and the PREADY signa
```

Generated SVAs

nl_plans.txt | property_goldmine_2.sva

AssertionForge > logs > gen_plan_2026-03-21T04-20-04.109787_Mac_dhruvrpansuriya > tbs > property_goldmine_2.sva

```
module apb_props #(
  parameter ADDR_WIDTH = 32,
  parameter DATA_WIDTH = 32
)(
  input  wire                    PCLK,
  input  wire                    PRESETn,
  input  wire [ADDR_WIDTH-1:0]   PADDR,
  input  wire                    PSEL,
  input  wire                    PENABLE,
  input  wire                    PWRITE,
  input  wire [DATA_WIDTH-1:0]   PWDATA,
  input  wire [DATA_WIDTH-1:0]   PRDATA,
  input  wire                    PREADY,
  input  wire                    PSLVERR
);

property a2;
@(posedge PCLK) (!PRESETn) ##6 ((PADDR >= 24) && (PADDR <= 31)) && (!PSLVERR);
endproperty
assert_a2: assert property(a2);

endmodule
```

Signal Mapping Output


```
{
  "canonical_name": "peripheral_select",
  "spec_mentions": [
    "PERIPHERAL SELECT"
  ],
  "rtl_name": "sdc_top.psel",
  "rtl_module": "sdc_top",
  "active_low": false,
  "confidence": 1.0,
  "match_method": "llm_alignment",
  "reasoning": "The signal psel is a select signal, which matches the PERIPHERAL SELECT mention in the spec."
},
{
  "canonical_name": "transfer_enable",
  "spec_mentions": [
    "TRANSFER ENABLE"
  ],
  "rtl_name": "sdc_top.penable",
  "rtl_module": "sdc_top",
  "active_low": false,
  "confidence": 1.0,
  "match_method": "llm_alignment",
  "reasoning": "The signal penable is an enable signal for transfer, which matches the TRANSFER ENABLE mention in the spec."
},
```

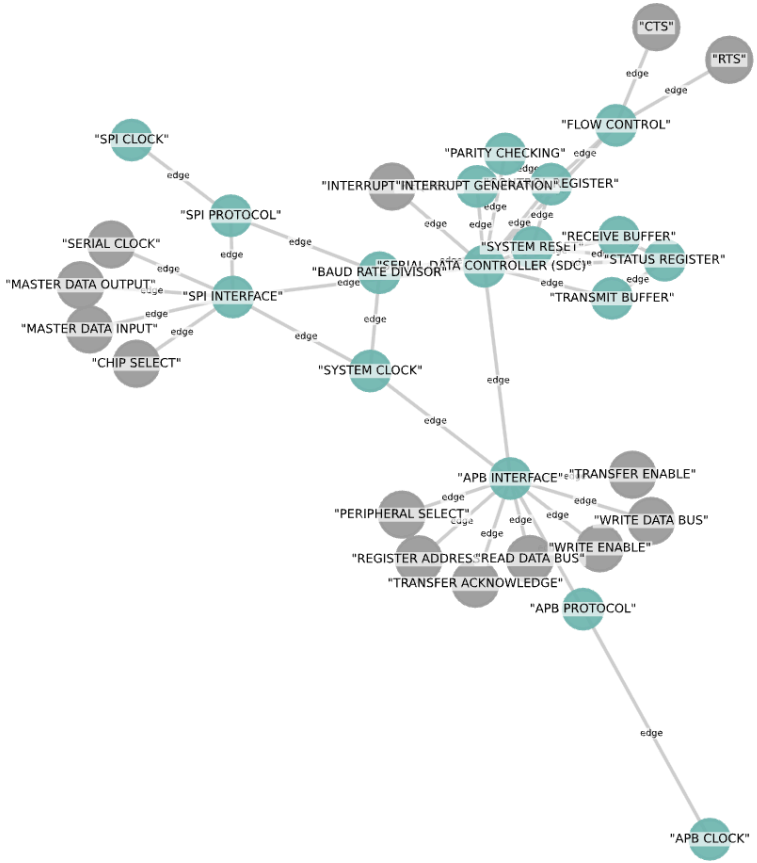
LLMLingua Token Reduction Summary

```
LLMLingua Token Reduction Summary
=================================

Overall Statistics:
  Tokens without LLMLingua : 4616
  Tokens with LLMLingua    : 3876
  Total Token Reduction    : 16.03%

Detailed Breakdown:
  Dynamic NL Generation:
    Origin: 2163, Compressed: 1807
  Static NL Generation:
    Origin: 0, Compressed: 0
  Dynamic SVA Generation:
    Origin: 2453, Compressed: 2069
  Static SVA Generation:
    Origin: 0, Compressed: 0
```

Verible Syntax Checking Results


```
12    {
13      "file_name": "bad_missing_endproperty.sva",
14      "status": "discarded",
15      "final_sva": "property p_bad3;\n  @(posedge PCLK) (PSEL) |-> ##1 (PREADY);\nassert property (p_bad3);",
16      "loop_a_repaired": false,
17      "loop_b_repaired": true,
18      "reason": "Loop B failed after 1 repair attempt (Verible syntax still invalid)\nFinal Verible error:\n/var/folders/gf
19      "invalid_signals_before_repair": [],
20      "syntax_error_before_repair": "/var/folders/gf/4trhcx9n3036fwzjjkf7gk1h0000gn/T/tmpibnvtqfh.sv:3:3: syntax error at t
21    },
22    {
23      "file_name": "bad_missing_semicolon.sva",
24      "status": "accepted",
25      "final_sva": "property p_bad1;\n  @(posedge PCLK) (PSEL && PENABLE) |-> (PRDATA == PWDATA)\nendproperty\nassert prope
26      "loop_a_repaired": false,
27      "loop_b_repaired": false,
28      "reason": "Accepted (passed Loop A + Loop B)",
29      "invalid_signals_before_repair": [],
30      "syntax_error_before_repair": ""
31    },
```

Visualizing Knowledge Graph



Addressing Core Challenges — P1 to P5 Summary

P1

Long-Context Limits

KG node-to-node linking reduces irrelevant chunks · SSR multi-resolution retrieval · LLMLingua minimal context selection (RQ1.1)

P2

Agent Memory

Unified KG as persistent cross-domain memory · structured relational schema vs flat embeddings (RQ1.4, RQ1.6)

P3

Cloud Communication Efficiency

LLMLingua: 16% token reduction · GPTCache eliminates redundant API calls · Groq for fast-turnaround agents (RQ1.7, RQ1.8)

P4

Redundancy in Multi-Agent Systems

GPTCache: early redundancy detection across agents (RQ2.2)

P5

Feedback-Driven Refinement

Verible closed-loop repair: binary + structured + localised error signals · bounded iterations · context preserved · syntax decoupled from semantics (RQ3.1–3.3)

Conclusion & Future Work

Real LLM Repair Integration

Implement repair_sva_with_llm() with schema-constrained JSON output for systematic, parseable fix generation.

Semantic & Formal Validation Stage

Add post-syntax validation: property equivalence checks, formal model-checking hooks (JasperGold), functional correctness beyond Verible.

Robust Signal Edge Materialization

Improve mention-matching robustness (fuzzy + semantic + edit-distance) for reliable corresponds_to edge creation even with naming noise.

Benchmark Suite

Quantitative evaluation: cost/latency/acceptance with vs without cache & multiagent; SVA correctness (#SynC, #SemC) across real-world hardware designs.

Thank you